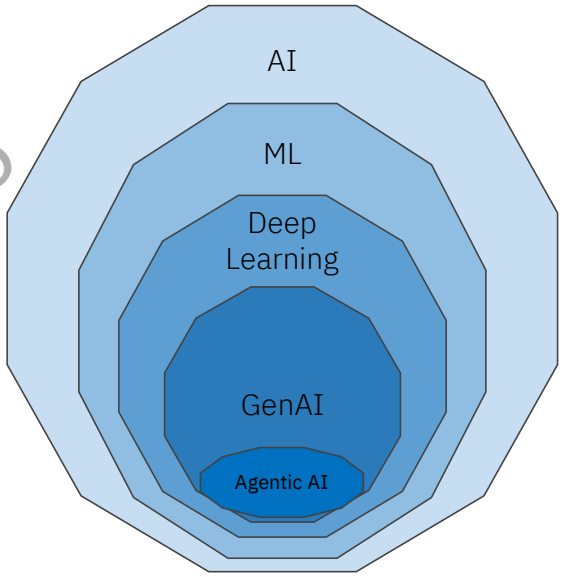
Introduction to AI

Week 3: Search Strategies

Nguyen Tuan Khang *PhD.* | khangnt@vn.ibm.com |
090 830 6668
Data & AI Leader, IBM ASEAN

Tran Dang An
Assistant

USTH 2025



1

Objectives

- Understand core search principles, including classical searches
- Apply traditional techniques to search information such as word search and regular expressions for pattern matching and full-text search
- Explain about knowledge search that uses vector-based semantic search approach (GenAI)
- Run some practical demo/labs to evaluate search queries and retrieval results.

USTH 2025

2

2

AI Hierarchy

Artificial Intelligence

W1: Basic concepts, history of AI
W2: Knowledge representation
W3: Search strategies

Machine Learning

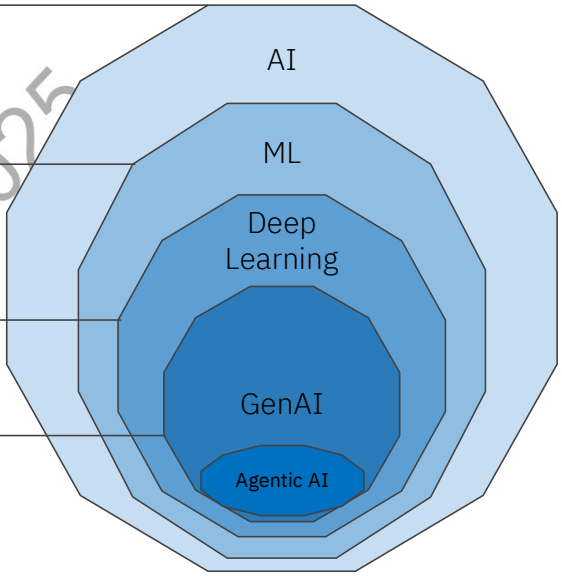
W4: Statistical learning
W5: Supervised machine learning
W6: Unsupervised machine learning

Deep Learning

W7: Perceptron learning and neural network learning
W8: Deep learning

Generative AI

W9 + W10: Modern AI and development in real world applications (LLM & GenAI)



USTH 2025

3

3

Open questions

- If you have three-digit lock, how do you crack the code?
- If you are lost a park (or worse, a maze), what is the best way to find the exit?
- In a world of exponential data growth, how can we quickly and accurately find content that matches our context and intent?

USTH 2025

4

4

Contents

- Search concepts
- Classical searches
- Content searches: word, regex and full-text
- Knowledge search: sematic search

USTH 2025

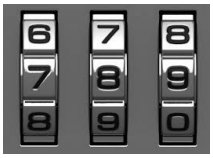
5

5

What is Search?

Search is a class of techniques for systematically finding or constructing solutions to problems.

- Example technique: generate-and-test.
- Example problem: find a combination to unlock key.
  - Generate a possible solution.
  - Test the solution.
  - If solution found THEN done ELSE return to step 1.



USTH 2025

6

6

Why Search?

A knowledge base stores facts, search transforms these facts into usable answers.

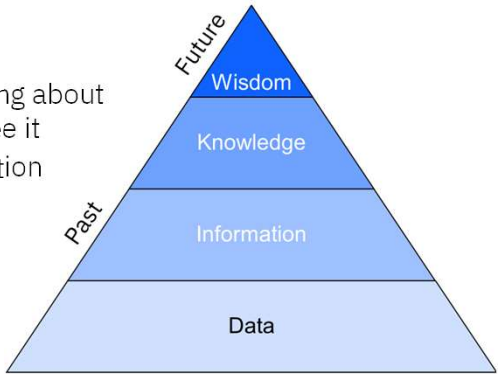
- Search is the fundamental technique of AI to explore knowledge and intelligence
  - Possible answers, decisions or courses of action are structured into an abstract space, which we then search.
- Many AI problems can be formulated as search problems: Path planning, Games, NLP, ML...
- Without fast, accurate retrieval, downstream AI applications (classifiers, generators, reasoners) has little relevant context to operate on.

7

7

Search in AI applications

- Classical searches (data search)
  - Uninformed: we move through the space without worrying about what is coming next, but recognizing the answer if we see it
  - Informed: we guess what is ahead and use that information to decide where to look next (heuristic)
- Content searches (information search): find information based on the actual meaning, text, or structure of the content itself



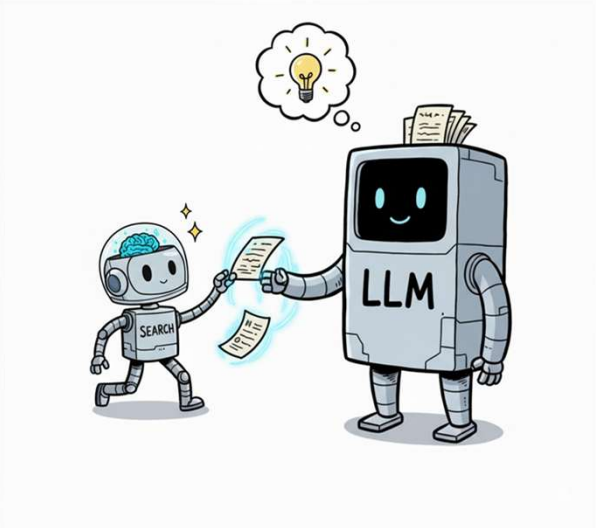
- Knowledge searches: retrieve relevant, meaningful knowledge from a knowledge base, rather than simply finding raw data or documents. It focuses on answers, explanations, insights, and reasoning, not just locating content.

8

8

Example search in GenAI Library

Analogy: A library without an index, messy, unstructured. Search finds the right book/pages to help LLM answer a question it doesn't know

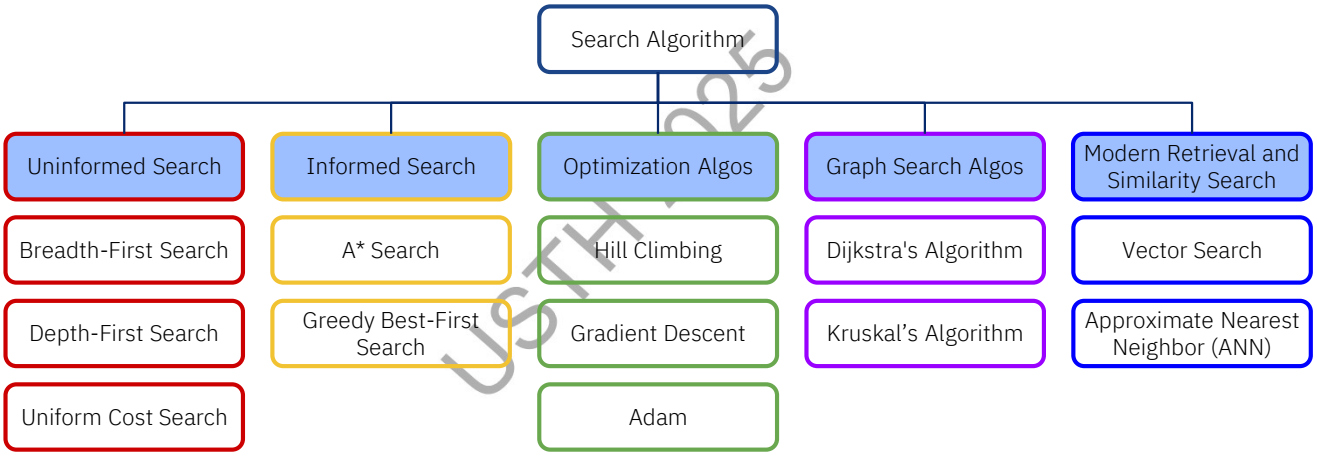


USTH 20

9

9

Search algorithms



USTH 2025

10

10

Quick history & timeline

1	1950s–1960s	Classical search and graph/tree algorithms (foundations for symbolic AI; problem solving as search).
2	1968	A* introduced (best-first with admissible heuristics) — crucial for combining cost and heuristic knowledge.
3	1990s–2000s	Full-text search and inverted indexes become industrial (search engines, Lucene/Elastic)
4	2010s	Dense vector embeddings & ANN accelerate semantic search (FAISS and HNSW popularized at-scale vector retrieval).
5	2020s	Retrieval-Augmented Generation (RAG) integrates retrieval with LLMs for grounded, up-to-date generation.

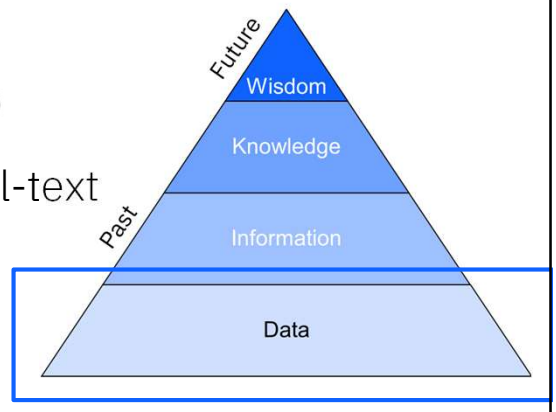
USTH 2025

11

11

Contents

- Search concepts
- Classical searches
- Content searches: word, regex and full-text
- Knowledge search: sematic search



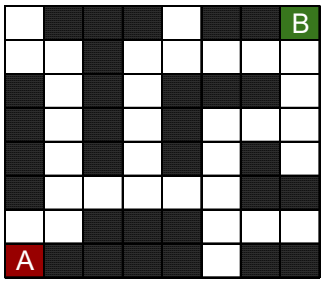
USTH 2025

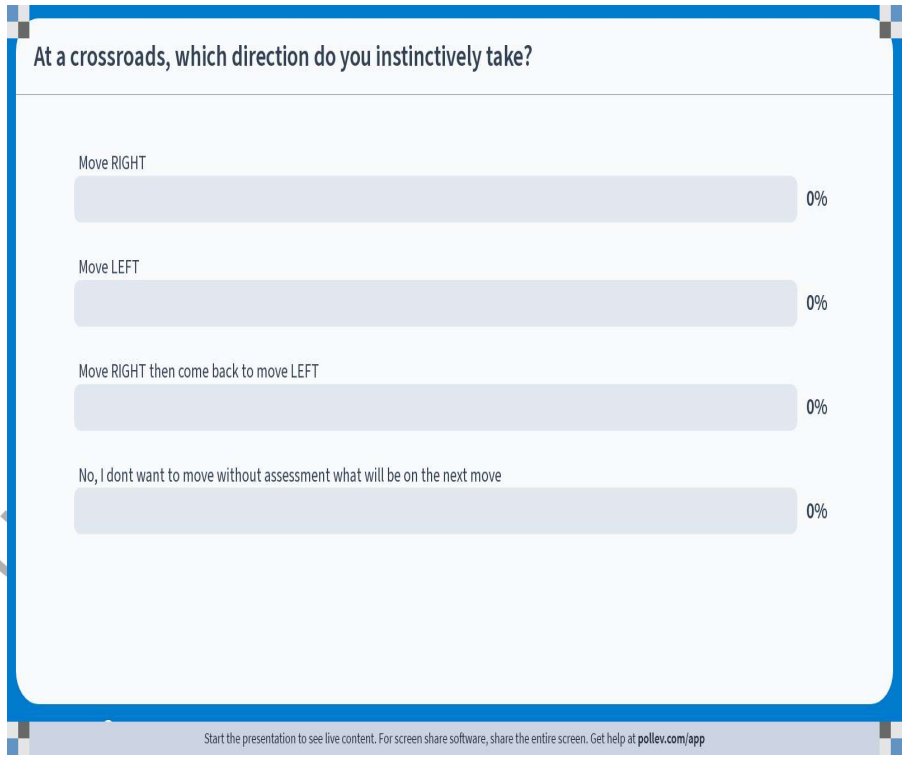
12

12

Quiz

- Get lost in maze



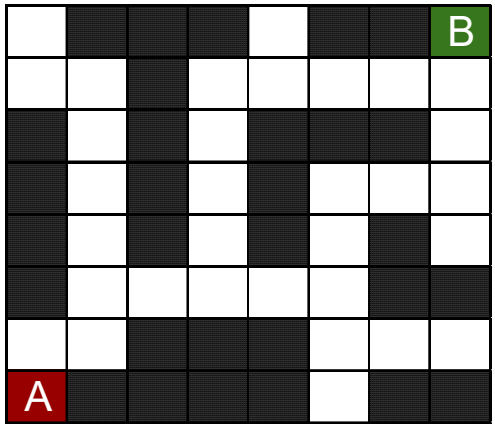


13

Problem statement

- Before we search, we must define the problem for our agent.

  - State: The agent's (X, Y) coordinate in the maze.
  - Initial State: The "Start" cell (cell A)
  - Actions: Move North, South, East, West (if not a wall).
  - Goal Test: Is the current state the "End" cell (cell B)?
  - Path Cost: Each move costs 1.



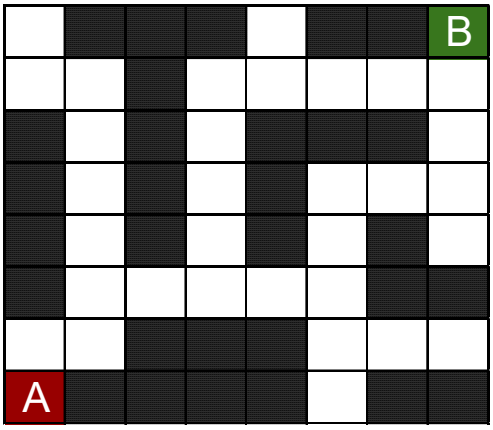
14

14

USTH 2025

Fundamental algorithms

- We'll explore some fundamental algorithms:
  - Blind search
    - Depth-First Search (DFS)
    - Breadth-First Search (BFS)
  - A better way with heuristic search (A* Search)



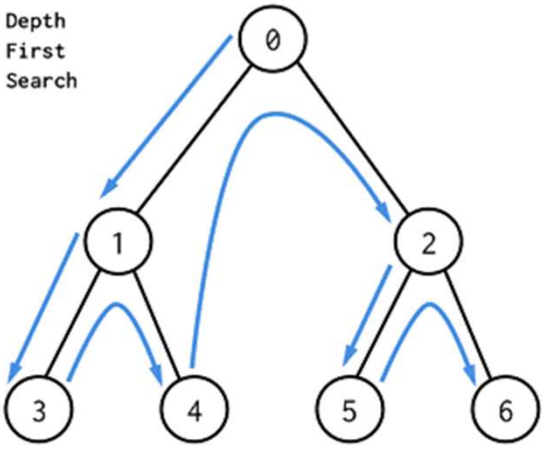
USTH 2025

15

15

Depth-First Search (DFS)

- Strategy: Go as deep as possible down one path. Only backtrack when you hit a dead end.
- Introspection: Always expands the newest node added to the Frontier.
- Data Structure: A Stack (Last-In, First-Out).
- Behavior in a Maze: Dives deep into one corridor, potentially taking a very long, winding route. It's fast but reckless.



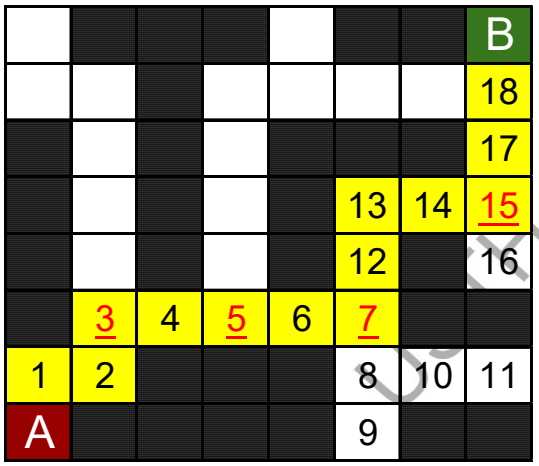
USTH 2025

16

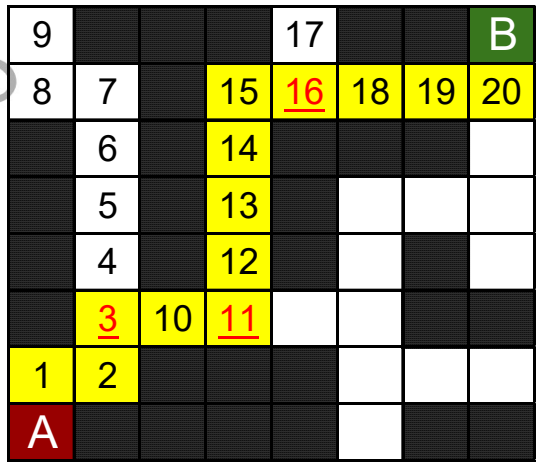
16

Depth-First Search (DFS)

Right first



Left first

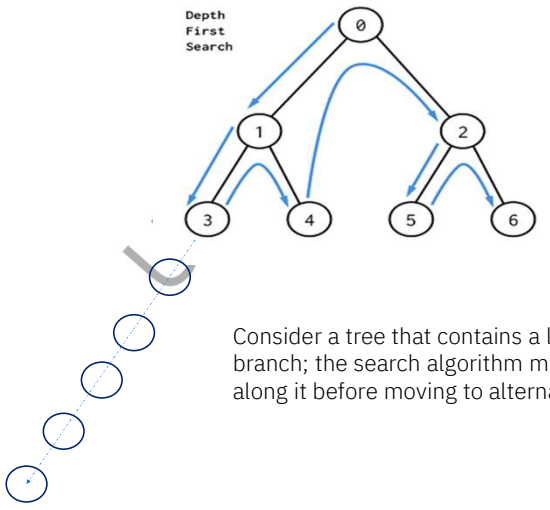


17

17

Example with special tree

- What happens if we want to search number 6?



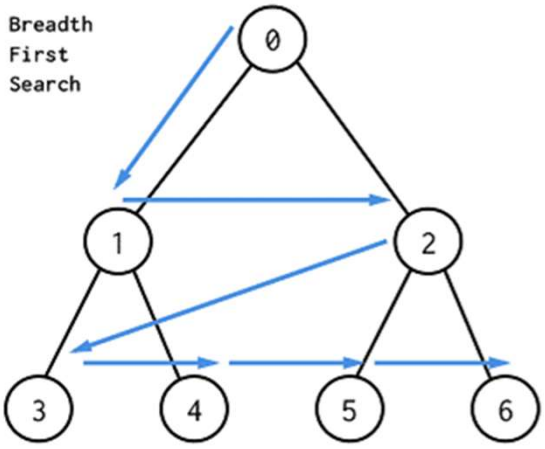
Consider a tree that contains a long, extended branch; the search algorithm must traverse deeply along it before moving to alternative branches

18

18

Breadth-First Search (BFS): The Expanding Wave

- Strategy: Explore all neighbors at the current depth before moving deeper.
- Introspection: Always expands the oldest node on the Frontier.
- Data Structure: A Queue (First-In, First-Out).
- Behavior in a Maze: Spreads out from the start like a wave, exploring in all directions equally. Guaranteed to find the shortest path in terms of steps.

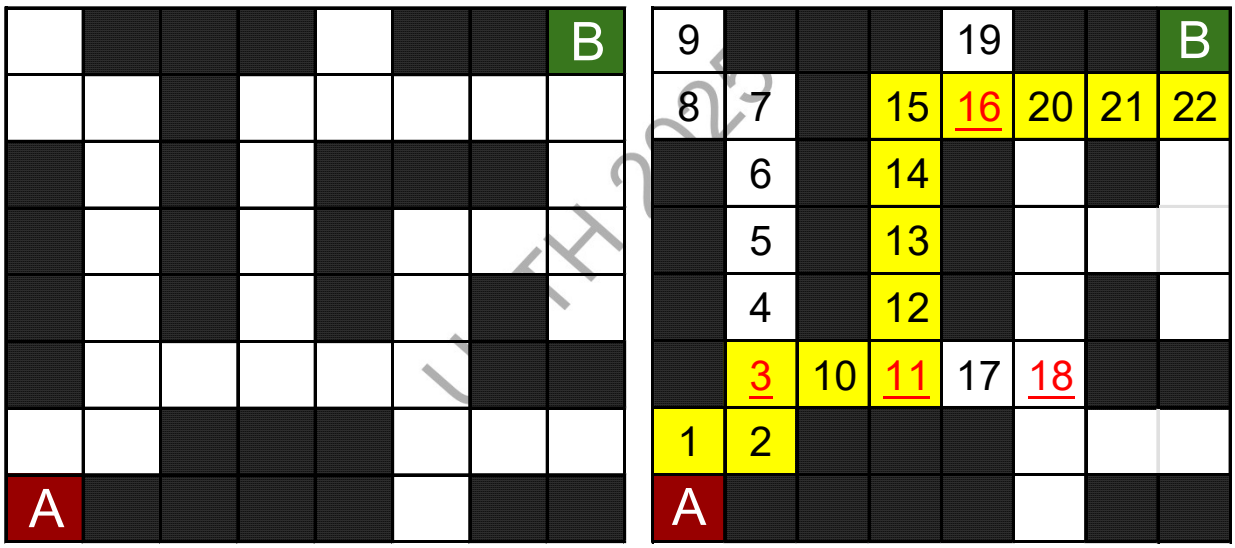


USTH 2025

19

19

Breadth-First Search (BFS)



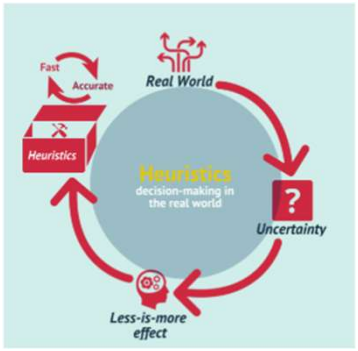
USTH 2025

20

20

A Better Way: Introducing the Heuristic

- DFS is reckless, and BFS is inefficient. Neither uses any information about the maze layout to make a smarter choice
  - They are blind.
- Let's give our agent a "sense of direction" thru smt called "guessing"
  - This is a heuristic.
- Heuristic Function **h(n)**: An estimate of the distance from the current cell to the goal/target cell.



USTH 2025

21

21

A* Search: The Smart Explorer

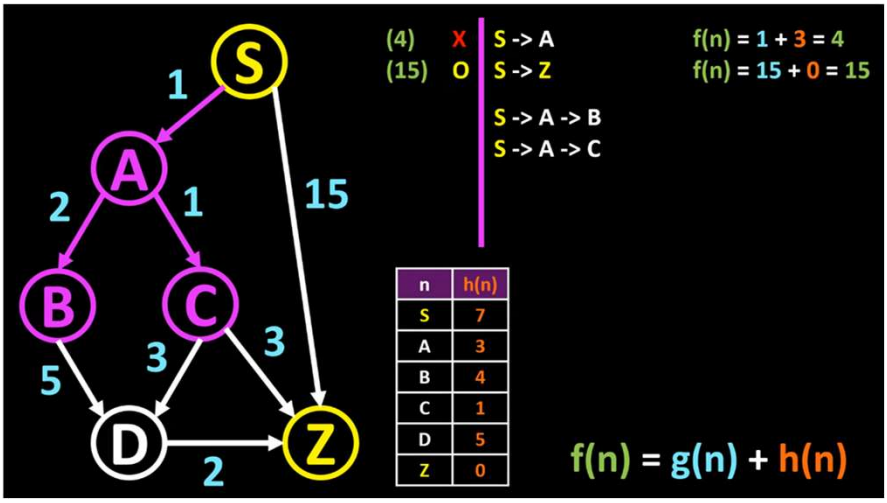
- Strategy: Combine what we know with our best guess.
- Introspection: Expands the node with the lowest value of **f(n)**.
- The A* Formula: **f(n) = g(n) + h(n)**
  - g(n): The actual cost (number of steps) from the start to the current cell **n**.
  - h(n): Our heuristic guess (steps or distance) from cell **n** to the goal/target call.
- A* balances finding a short path (**g(n)**) with making progress toward the goal (**h(n)**).

USTH 2025

22

22

Visualization



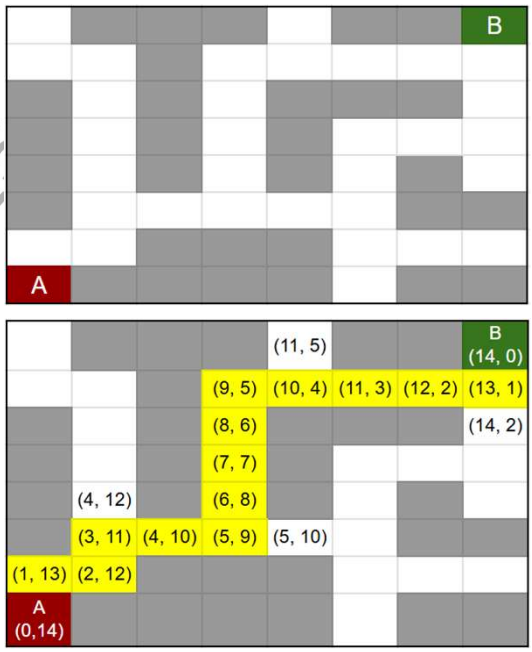
23

23

A* Search: The Smart Explorer

Imagine you're at 'A' and want to reach 'B'.

1. You keep a "to-do" list of squares to check, prioritized by a score.
2. You add your first possible steps (neighbors) to this list.
3. Each square's score is calculated:
   Score = g(Steps already taken) + h(Best guess to 'B')
4. You always choose to explore the square on your entire list that has the lowest score.
5. When you explore that square, you add its neighbors to the list, calculate their scores, and repeat.

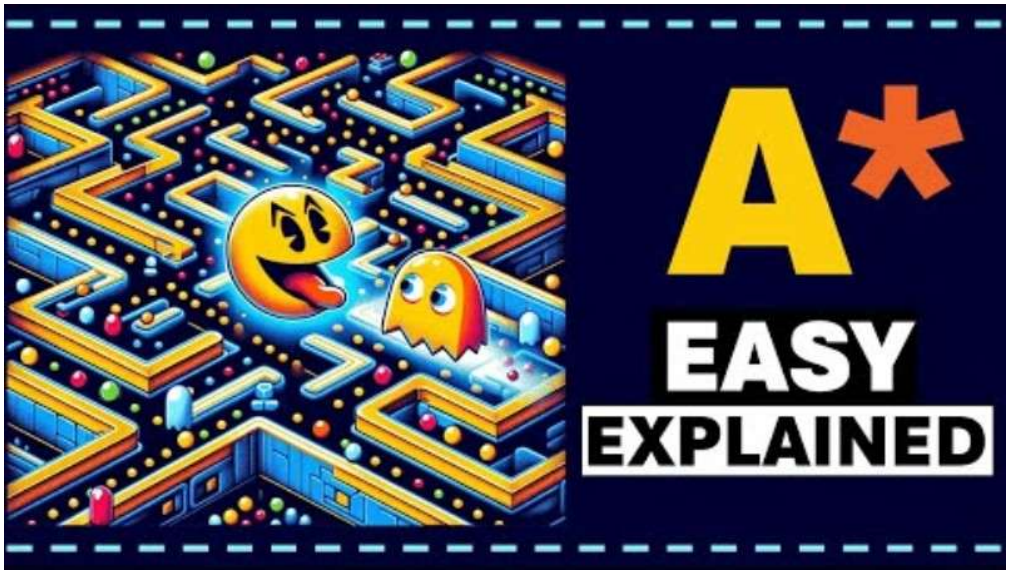


USTH 2025

24

24

A* Search explained



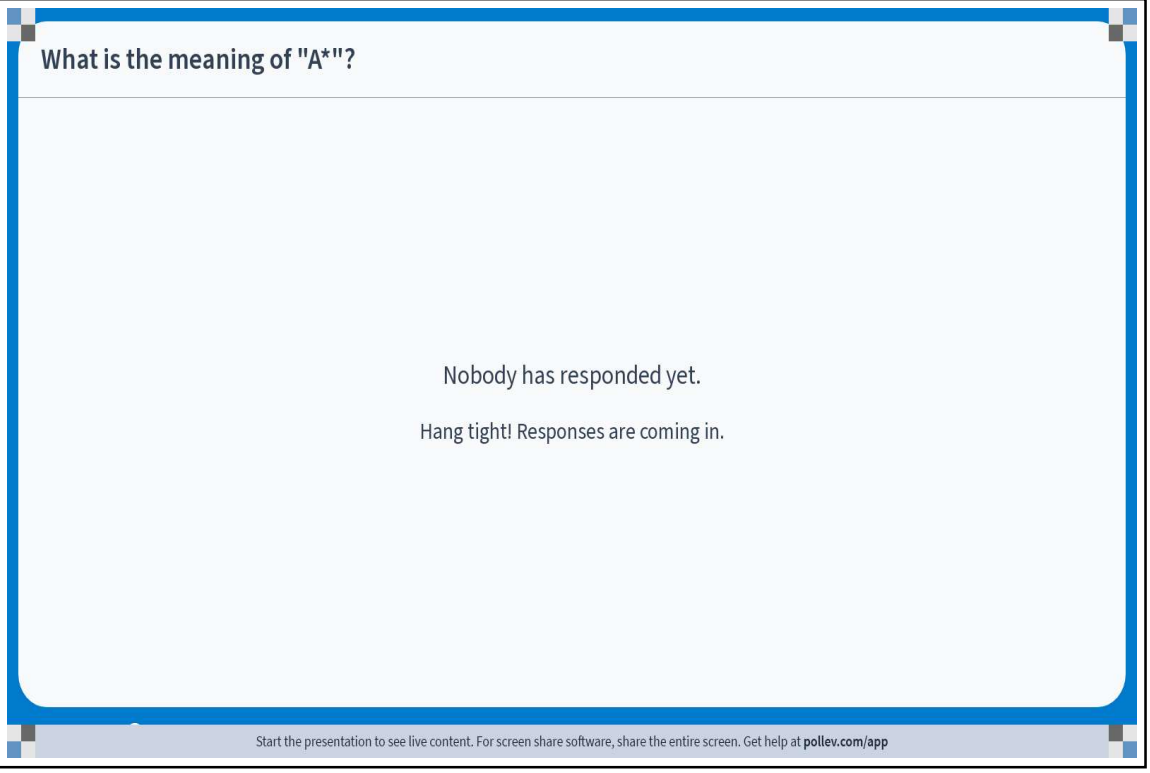
https://www.youtube.com/watch?v=kEY1OxOj_CY

25

25

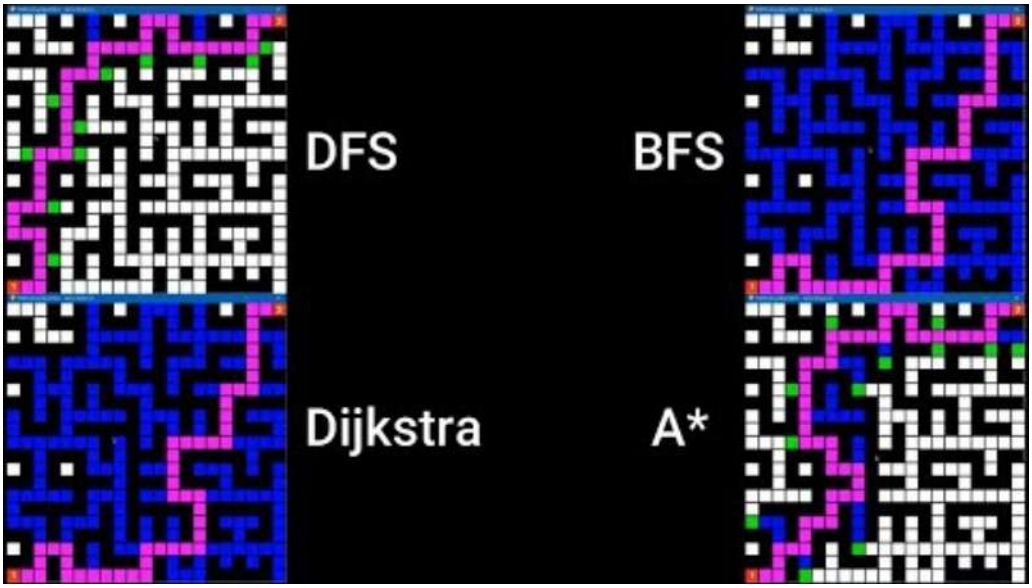
Quiz

- A*???



26

Classical searches in summary



https://www.youtube.com/watch?v=aW9kZcJx64o

27

27

Comparison

Parameter	Uninformed Search	Informed Search (Heuristic Search)
Core Strategy	Uses no problem-specific knowledge to guide the search process.	Uses domain-specific information, in the form of a heuristic function, to make a "best guess" about which path to take.
Analogy	A blindfolded explorer, systematically tapping every wall to find a way out.	An explorer with a compass that always points toward the goal.
Performance	Finds a solution slowly compared to informed search.	Finds a solution much more quickly by eliminating unpromising paths.
Cost Factor	The cost in terms of time and memory is high because many nodes must be explored.	The cost is low because fewer nodes are expanded to find the goal.
Examples	Breadth-First Search (BFS), Depth-First Search (DFS).	Greedy Best-First Search, A* Search.

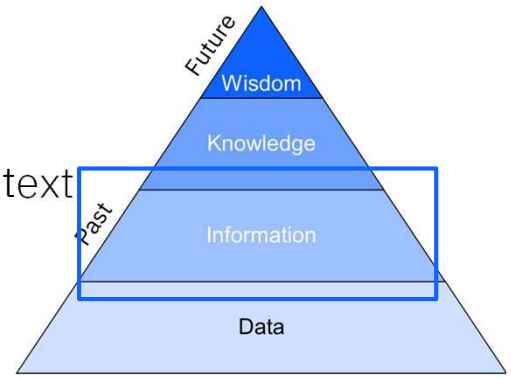
USTH 2025

28

28

Contents

- Search concepts
- Classical searches
- Content searches: word, regex and full-text
- Knowledge search: sematic search



USTH 2025

29

29

Content Search: Beyond the Maze

Topics We'll Cover:

➔Approach 1: Simple Word Match (The Naive Way)

➔Approach 2: Pattern Matching with Regex (A Smarter Way)

➔Approach 3: Full-Text Search (The Real-World Applications)

USTH 2025

30

30

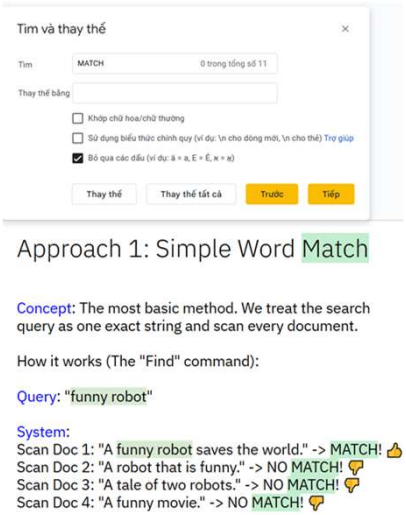
Approach 1: Simple Word Match

Concept: The most basic method. We treat the search query as one exact string and scan every document.

Query: "funny robot"

How it works: The "Find" command

System:
Scan Doc 1: "A funny robot saves the world." -> MATCH! 👍
Scan Doc 2: "A robot that is funny." -> NO MATCH! 👎
Scan Doc 3: "A tale of two robots." -> NO MATCH! 👎
Scan Doc 4: "A funny movie." -> NO MATCH! 👎



USTH 2025

31

31

Approach 1: Simple Word Match - Problems

- It's too rigid
  - It fails on "funny robots" (plural) or "robot that is funny" (word order).
- It's painfully slow
  - Imagine reading every book in a library from cover to cover just to find one word.
  - It's an O(M x N) problem (where N is documents, M is text length).
  - It cannot scale to millions of titles.

USTH 2025

32

32

Approach 2: Pattern Matching (Regex)

Concept: A big step up. Instead of a fixed string, we use Regular Expressions (Regex) to search for a pattern.

Query: "funny robot"

How it works: Regex Pattern

We could write `funny.*robot|robot.*funny`

This means:
1. find 'funny', then anything, then 'robot'
OR
2. find 'robot', then anything, then 'funny'

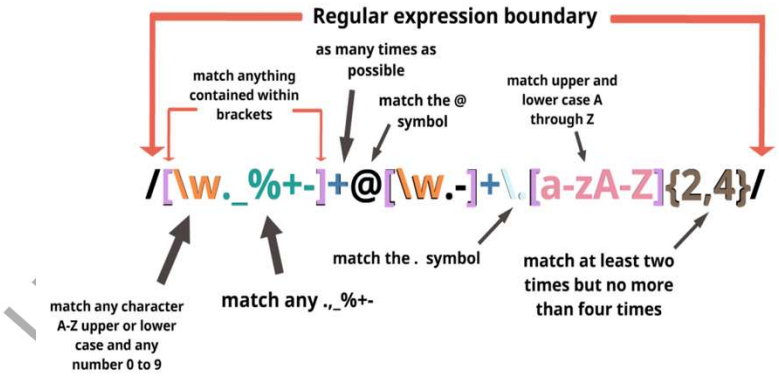
System:
Scan Doc 1: "A funny robot saves the world." -> MATCH! 👍
Scan Doc 2: "A robot that is funny." -> MATCH! 👍
Scan Doc 3: "A tale of two robots." -> NO MATCH! 👎 (We could fix this with robots?)

33

33

Example of validating email address with Regex

- john@doe.com
- john@doe.com1
- john@doe.com.vn.vn.vn
- john@doe.something
- 123john@doe.com
- John----@doe.com



https://4geeks.com/lesson/regex-tutorial-regular-expression-examples

34

34

Regex Explained



https://www.youtube.com/watch?v=sXQxhojSdZM

35

35

Approach 2: Pattern Matching (Regex) - Problems

- Still too slow
  - Regex is powerful, but running complex patterns against millions of documents in real-time is still a massive performance bottleneck.
- No understanding of relevance
  - It can't tell how important the match is. A movie titled "Funny Robot" is more relevant than a movie that has "funny robot" in one review, but Regex treats them the same.
- Even with a “magic glass” this time to read contents, We're still reading every document

USTH 2025

36

36

Approach 3: Full-Text Search

- Concept: We need to stop "scanning" and start "looking up". This is what search engines like Google, Netflix, and Amazon use.
- The Question: How can we possibly get this speed and intelligence?
- The Answer: We don't search the documents at all.
  - We build an index first.

USTH 2025

The Inverted Index

- The map that makes search instant.
- Lists every word and where it lives.
- Lets the engine look up answers, not scan for them.

37

37

Full-Text Search – The Goal

What we actually want:

1. Speed: Results must be instantaneous (sub-second).
2. Relevance (Ranking): Don't just show matches; show the best matches first. A match in the Title is more important than a match in the Description.
3. Linguistic Awareness: The system should be smart.
   ➔It should handle stemming (e.g., running -> run).
   ➔It should ignore stop words (common words like a, the, and).
   ➔It should handle plurals, synonyms, and typos.



USTH 2025

38

38

Full-Text Search – The Engine

The Analogy: A Textbook Index

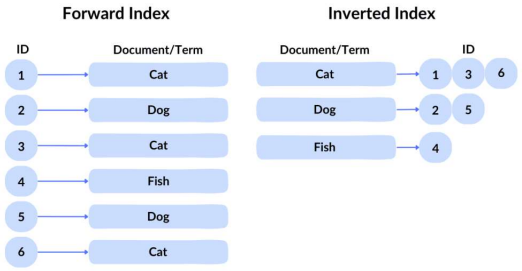
How do you find “learning" in our textbook “Artificial Intelligence - A Modern Approach”?

You don't read all pages, lets check Index section of the book





An Inverted Index is exactly this, but for all words and all documents.



USTH 2025

39

39

Example: Building The Index

Step 1: Get Documents (Movie Titles)

Doc 1: "Star Wars: A New Hope"
Doc 2: "Star Trek: The New Generation"
Doc 3: "A Star is Born"

Step 2: Tokenize & Normalize (Split into words, lowercase, remove stop words)

Doc 1: star, wars, new, hope
Doc 2: star, trek, new, generation
Doc 3: star, is, born -> star, born (removed "is", "a", "the")

Step 3: Create the Index (The Map) This maps words (Tokens) to the documents that contain them.

Word	Document List
star	{Doc 1, Doc 2, Doc 3}
wars	{Doc 1}
new	{Doc 1, Doc 2}
hope	{Doc 1}
trek	{Doc 2}
generation	{Doc 2}
born	{Doc 3}

USTH 2025

40

40

Example: Searching with Index

Doc 1: "Star Wars: A New Hope"
Doc 2: "Star Trek: The New Generation"
Doc 3: "A Star is Born"

Word	Document List
star	{Doc 1, Doc 2, Doc 3}
wars	{Doc 1}
new	{Doc 1, Doc 2}
hope	{Doc 1}
trek	{Doc 2}
generation	{Doc 2}
born	{Doc 3}

Tokenize Query: new, star

1. **Consult the Index**: Lookup words to get the relevant document
   ➔Look up new: {Doc 1, Doc 2}
   ➔Look up star: {Doc 1, Doc 2, Doc 3}

2. **Find the Intersection**: Which documents are in both lists?
   ➔{Doc 1, Doc 2} ∩ {Doc 1, Doc 2, Doc 3} = {Doc 1, Doc 2}

3. **Rank & Return**:
   ➔The system returns "Star Wars: A New Hope" and "Star Trek: The New Generation".
   ➔It might rank "A Star is Born" (Doc 3) lower because it only matched one word.

USTH 2025

The advantage: We never scanned the original documents, just did two quick lookups in our index and compared two short lists. This is why it's instant.

41

41

Full-Text Search Explained



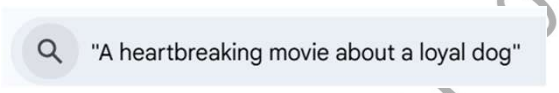
https://www.youtube.com/watch?v=hhUBfFX6N9g

42

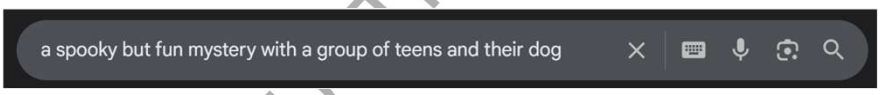
42

Approach 3: Full-Text Search - Problems

- The Full-Text Search is incredibly powerful, but it has a fundamental limit: it only understands keywords.
- What if you search for:



It can't find Hachi: A Dog's Tale unless the exact word "heartbreaking" is in the text



How would it know to find Guardians of the Galaxy or Deadpool? That's a vibe, not a keyword

43

43

Summary: From Simple Match to Smart Search

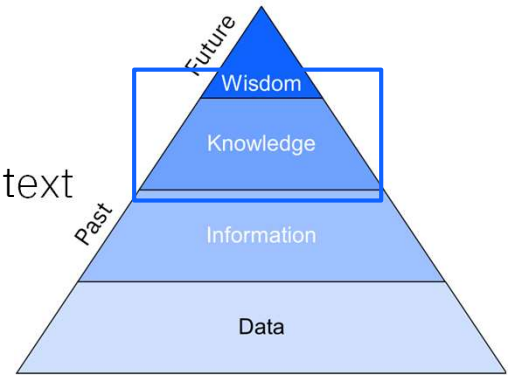
- Simple Word Match: Too simple, far too slow. Fails to find relevant results.
- Pattern Matching (Regex): More flexible but still requires scanning every document. Not scalable.
- Full-Text Search: The real-world solution as it's fast, relevant, and smart by using the Inverted Index: The engine that powers full-text search. It's a map from words back to the documents that contain them.

44

44

Contents

- Search concepts
- Classical searches
- Content searches: word, regex and full-text
- Knowledge search: sematic search

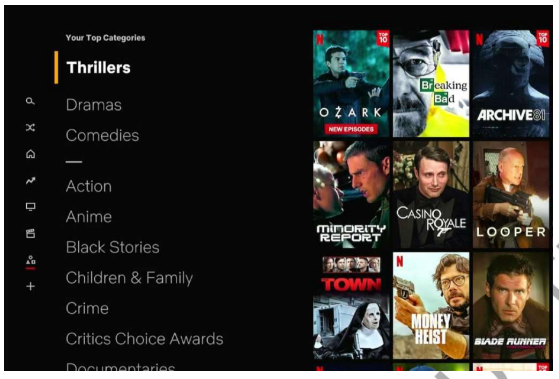


USTH 2025

45

45

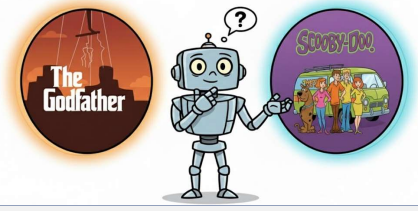
More than Text Search



Search on Netflix

a spooky but fun mystery with a group of teens and their dog

The Challenge: How does the system instantly sift through all that data to find Scooby-Doo ?
and not The Godfather or Guardians of the Galaxy?



This isn't just about finding an exact match.
It's about understanding intent and relevance.

46

46

Quiz

- Do you know about the Netflix's challenge to identify the best algorithm to find the most suitable movie for users?

USTH 2025

47

47

Knowledge Searching in the age of GenAI



How would it know to find Scooby-Doo?
That's a vibe, not a keyword

This is a search for semantic meaning, not just matching words.



48

48

Knowledge Searching in the age of GenAI

- To solve this, we need to go beyond keywords and teach the computer what words mean. This is the world of Semantic Search with vector embeddings and GenAI

- We will continue to dive deeper in W10 with “Knowledge Searching in the age of GenAI”



USTH 2025

49

49

Thank you!

USTH 2025

50

50